ES-FA / BTP Progress Report

Event-Driven SNN FPGA Accelerator

Updated May 5, 2026

Short Summary

The project has moved beyond the initial proposal stage. At this point, ES-FA is a local working pipeline with SNN training, hardware-aware estimates, experiment runs, analysis plots, RTL blocks, xsim/Vivado validation scripts, a small CLI layer, and LaTeX documentation. The main unfinished item is physical validation on the KV260 board. Local xsim/Vivado evidence already exists, but board-level measurements still have to be added.

1 Project Aim

The idea behind ES-FA is to study whether sparse spiking neural network activity can be used to reduce the work done by an FPGA accelerator. The project therefore tracks both model quality and hardware-facing cost. Accuracy alone is not enough; the run also records spike activity, memory-access estimates, energy proxy, latency proxy, and hardware validation data where available.

In simple terms, the work asks:

- Can a small SNN be trained and exported in a reproducible way?
- Can dense and event-driven execution be compared using the same metrics?
- Can the RTL and validation flow produce local evidence for the KV260 path?
- Can the project be demonstrated as a runnable system instead of a loose set of scripts?

2 Work Completed So Far

Area	Status
Training	Baseline LIF SNN training is available for a `784 -> 128 -> 64 -> 10` network.
Estimators	Spike cost, memory access, energy proxy, and latency proxy functions are connected to training.
Experiments	Hardware-aware loss and adaptive dataflow runs are present in the current results.
Exports	Metrics, histories, manifests, checkpoints, INT8 weights, spike stats, and raster files are generated.
RTL	Router, scheduler, event queue, BRAM, LIF PE, and top-level Verilog sources were preserved.
Validation	KV260-oriented xsim and Vivado batch scripts generate local logs, reports, and parsed JSON.
System layer	Intent engine, task router, SNN/math/control modules, adaptive policy, and CLI are implemented.
Docs	Setup guide, manual, architecture notes, checkpoints, paper source, and this report are now in the repo.

3 Existing Work Preserved

The project was not rebuilt from scratch. The following parts were kept because they were already useful and working:

- `paper1_training/`: current SNN model, training core, baseline script, and export utilities.
- `paper2_hardware_model/estimators.py`: hardware proxy functions.
- `experiments/`: proposal experiment folders and configs.
- `iterations/`: refinement runs.
- `analysis/compare.py`: ranking and plotting workflow.
- `hardware/`: Verilog RTL and testbenches.
- `software/`: earlier train/export/bank-mapping scripts.

4 What Was Added or Improved

The newer work mainly wraps the existing pieces into a cleaner local pipeline:

- run manifests and layer-activity exports were added so results are easier to trace;
- `hardware_validation/kv260/` was added for xsim regression, Vivado Tcl builds, report parsing, and hardware metric collection;
- `system_layer/` was added for intent classification and routing;
- `deployment/` was added for the CLI and adaptive runtime policy;

- `analysis/evidence_report.py` was added for baseline-vs-optimized tables and interpretations;
- presentation-facing LaTeX documents were added under `docs/` and `paper_output/`.

5 Current Architecture

The current repo has four practical layers:

```
Software:
  SNN training -> hardware estimators -> experiments -> analysis

Hardware:
  Verilog RTL -> xsim regression -> Vivado build -> parsed reports

Runtime:
  intent engine -> router -> SNN/math/control module -> adaptive policy

Outputs:
  metrics -> plots -> evidence tables -> LaTeX reports/PDFs
```

The SNN model used in the current baseline is:

```
MNIST image: 28 x 28
Flattened input: 784
Hidden layer 1: 128 LIF neurons
Hidden layer 2: 64 LIF neurons
Output layer: 10 classes
Default temporal window: 16 time steps
```

The FPGA-side flow being validated is:

```
input events -> spike router -> scheduler/event queue
  -> weight/neuron memory -> LIF processing element
  -> writeback and counters
```

6 Software Results

The current local ranking compares the baseline with two experiment outputs:

Run	Accuracy	Sparsity	Energy proxy	Score
exp1_hardware_aware_loss	0.9570	0.5648	4,592,863.17	0.7699
baseline_paper1	0.9570	0.5648	22,604,295.76	0.7387
exp5_dataflow_adaptation	0.9570	0.5648	45,016,894.73	0.6699

The strongest current software result is `exp1_hardware_aware_loss`. In the current run, it keeps the same accuracy as the baseline while reducing the energy proxy by about 79.68 percent against the dense baseline estimate. This is still a proxy result, so it should not be presented as measured board power.

7 Hardware Validation

The hardware validation path is local and KV260-oriented. It runs:

1. xsim regression for dense and event scheduler modes;
2. Vivado synthesis/implementation through `hardware_validation/kv260/vivado/build.tcl`;
3. parsing of utilization and timing reports;
4. merging of xsim, Vivado, model, mode, and estimator fields into `hardware_metrics.json`.

Current local hardware evidence exists for baseline and adaptive variants in dense and event modes. The current parsed results show:

- latency: 5760 ns in the active validation window;
- cycle count: 576 cycles;
- LUT usage: 22,193 in the parsed reports;
- estimator-vs-measured latency MAPE: 33.33 percent across four hardware runs.

The dense and event modes currently report equal latency/cycle count in the generated evidence table. That result is useful because it keeps the comparison honest, but it also shows what needs work next: the sparse event path needs stronger hardware stimuli and real KV260 board measurements.

8 System and CLI Layer

The system layer was added so the project can be shown through commands, not only through training scripts. The CLI currently supports:

```
python deployment/cli_interface/main.py --query "hi"
python deployment/cli_interface/main.py --query "strike rate 167.55 balls 39"
python deployment/cli_interface/main.py --query "classify results/runtime/sample_signal.
    bin"
```

Internally, the intent engine classifies a request as control, math, or SNN execution. The router sends it to the matching module. For SNN-style requests, the adaptive policy estimates signal sparsity and chooses dense or event mode. Every adaptive decision is logged in:

```
results/runtime/adaptive_decisions.jsonl
```

In the current sample signal, the adaptive path selects dense mode because the estimated sparsity is about 0.0039, below the 0.9000 threshold.

9 Documentation Added

The documentation now has a more usable shape:

- `docs/first_time_user_setup_guide.tex`: practical setup guide;
- `docs/architecture.md`: current architecture notes;

- `docs/checkpoints.md`: milestone and run-history ledger;
- `docs/manual.tex`: short operating manual;
- `paper_output/progress_report.tex`: this report;
- `paper_output/final_paper.tex`: paper-style draft source.

10 Open Work

- Run physical KV260 measurements and merge them into the same hardware metrics schema.
- Tune estimator constants using measured hardware data.
- Expand the hardware validation matrix beyond the current baseline and `exp5_dataflow_adaptation` variants.
- Improve event-mode test stimuli so sparse activity is exercised more clearly.
- Revisit timing closure where Vivado reports negative slack.
- Keep generated tool outputs out of the repo root when preparing the GitHub version.

11 Next Steps

1. Boot the KV260 with the prepared Kria Ubuntu SD card.
2. Re-run the smoke pipeline before the presentation to refresh visible outputs.
3. Run xsim-only validation if Vivado implementation time is limited.
4. Run full Vivado validation for baseline and adaptive variants when time is available.
5. Add board-side measurements to `results/hardware_validation/`.
6. Use those measurements to improve the estimator-vs-hardware match.

12 Final Note for Review

The project is now in a demonstrable state, but it should be described carefully: the software pipeline, plots, CLI, RTL, and local validation scripts are working; the final board measurement stage is still pending. The current evidence supports the ES-FA direction, especially the hardware-aware software result, but the final claim should wait for KV260 physical measurements.